

Leant2

Native Dependent Program Synthesis in Lean

Architecture, main algorithms,
trust boundaries, and executable experiments

Central proposal. Make exact Lean expressions, local contexts, and universes the semantic foundation. Search jointly for a program and the evidence that it satisfies its contract. Use domain-specific abstractions to guide search, never to replace the final Lean statement or its verification.

Design target	Lean implementation; Lean-only frontend; no Haskell type approximation as the authoritative representation.
Repository baseline	Leant <code>6bf05ad78c46</code> ; its Djex dependency <code>e237e8667190</code> .
Experimental target	Lean 4.34.0, checked remotely through AXLE using Wolfram HTTP transport.
Evidence included	Native search prototype; 27 synthesized affine-fold contracts with universal Lean proofs; negative controls; reproducible sources and normalized receipts.
Status	Architecture proposal with bounded prototypes, not a completed Leant2 implementation or a measured practical-coverage claim.

Abstract

Leant and Djex already combine type-directed search, behavioral checks, candidate provenance, and host-language replay. Their Lean-facing synthesis nevertheless passes through a separate synthesis type language. This article designs a successor whose authoritative goals, candidates, universe constraints, and local assumptions remain native Lean objects. The objective is not to decide arbitrary dependent-type inhabitation. It is to combine several bounded but complementary construction methods without introducing an approximation boundary between the language being searched and the language ultimately checked.

The proposed system has a Lean frontend and project-local workers; a frozen query boundary; an exact, context-indexed graph of program and proof holes; focused introduction and elimination; demand-driven polymorphic provider application; dependent case analysis; termination-aware recursion sketches; contract decomposition; counterexample-guided domain search; and fair, resource-bounded orchestration. Final acceptance checks the original statement in a controlled environment, audits assumptions, and separately checks executable-code requirements. Abstract types, observational signatures, and solver results are advisory unless accompanied by an explicitly justified pruning or reconstruction rule.

We give concrete search rules, scheduling and cache invariants, dependency-aware backtracking algorithms, contract and recursion interfaces, conditional soundness and relative-completeness statements, worked dependent examples, and an implementation sequence with acceptance gates. A small Lean-native prototype is compiled against Lean 4.34.0. Its final combined file audits twenty declarations: nineteen have no axioms and one uses only propositional extensionality. A separate reproducible experiment solves twenty-seven affine-fold specifications; a generic inductive certificate and all twenty-seven instantiated universal contracts are checked in Lean. Four negative controls distinguish ordinary compilation from proof acceptance. These experiments establish specific mechanisms, not production scalability or superiority over the existing tools.

How to read the evidence

Throughout, *proposed* describes Leant2 functionality; *implemented* describes only files in the accompanying prototype or experiment directories; and *checked* describes an actual reported Lean run. The mathematical guarantees in Section 11 are proved under stated architectural assumptions; they are not claims that the prototype implements every assumption. The affine-fold sufficiency theorem is formalized, while the general five-point necessity argument is proved in the article but not formalized in the supplied Lean file.

The remote check service returned the requested Lean version and executor identity, but not an exact Mathlib commit. No local Lean installation or independent external kernel checker was available in this session. The supplied receipts are explicitly labeled normalized transcriptions of observed responses, not raw HTTP captures. Sources and commands are supplied for independent reruns.

Contents

Abstract	i
How to read the evidence	i
1 The design decision	1
1.1 What the present repositories already provide	1
1.2 A practical scope, not a percentage promise	1
2 The query and result contracts	2
2.1 Freeze the question before searching	2
2.2 Behavioral inputs are Lean propositions	2
2.3 Statuses that cannot be conflated	3
2.4 Trust profiles and computational requirements	4
3 System architecture and semantic ownership	4
3.1 A Lean controller and project-matched workers	4
3.2 One semantic owner, several derived views	5
3.3 Persist plans, not dangling metavariables	5
3.4 Acceptance is a transaction	5
4 The native search state	6
4.1 A dependent AND–OR graph	6
4.2 Continuation-aware backtracking	6
4.3 Which goal should be solved next?	7
4.4 Freezing and reopening program choices	7
5 The provider inventory and type-level search	8
5.1 Retrieve a bounded relevant inventory	8
5.2 Instantiate full dependent spines	8
5.3 Type arguments are terms too	8
5.4 Dictionary evidence has computational identity	9
6 Focused native construction	9
6.1 Rules, priorities, and side conditions	9
6.2 Normalization is controlled work	10
6.3 Application search without premature argument choices	10
6.4 When proof automation helps and when it does not	11
7 Dependent elimination and indexed data	11
7.1 Construct the motive from the whole telescope	11
7.2 Impossible branches require evidence	11
7.3 Equality transport is an explicit search action	12
7.4 Elimination restrictions are part of the language	12
7.5 Worked indexed-constructor trace	12
8 Recursive program synthesis	13
8.1 A ladder of recursion methods	13
8.2 Generate programs and induction obligations together	13
8.3 Infer a useful carrier before enumerating a large body	14
8.4 Accumulator generalization and strengthened contracts	14
8.5 Well-founded recursion as an explicit later capability	14
8.6 Compile the form that will actually be exported	15
9 Contract-directed search and semantic domains	15
9.1 Search with the specification, not only after it	15
9.2 A domain adapter has four proof-relevant boundaries	15
9.3 Backward contract decomposition	16
9.4 Counterexample-guided refinement with three-valued verification	16
9.5 External solvers are optional and untrusted	17

9.6	Observational equivalence is not semantic equivalence	17
10	Fair search, abstraction refinement, and caching	17
10.1	Portfolio lanes with an explicit audit route	17
10.2	A finite grammar at every cost bound	18
10.3	Type-guided abstraction refinement as retrieval, not replacement	18
10.4	Cache keys include semantic context	18
10.5	Parallelism and determinism	19
11	Guarantees and their exact assumptions	19
11.1	Acceptance soundness	19
11.2	Termination of a bounded audit run	20
11.3	Relative completeness by dovetailing	20
11.4	Justified pruning	21
11.5	What a negative result can mean	21
12	An exact contract domain: affine list folds	21
12.1	Candidate schema and specification family	22
12.2	Why this domain fits Leant2	23
12.3	The executable experiment	23
12.4	Lean proof reconstruction	23
13	Native Lean experiments and verification results	24
13.1	What was implemented	24
13.2	The final native corpus	25
13.3	Remote compilation protocol	25
13.4	A useful failed attempt: recursor versus executable form	25
13.5	Negative integrity controls	26
13.6	What the evidence does not establish	26
14	Concrete implementation organization	27
14.1	Module boundaries	27
14.2	Rule and worker interfaces	27
14.3	A complete finalization algorithm	28
14.4	Initial operational profiles	28
14.5	Errors should explain the missing mechanism	29
15	A development sequence with measurable gates	29
15.1	Stage I: freeze, construct, and reject correctly	29
15.2	Stage II: native dependency as a first-class feature	29
15.3	Stage III: dependent elimination and structural recursion	29
15.4	Stage IV: contract domains and proof integration	30
15.5	Stage V: scale and assurance	30
15.6	Stage VI: broaden deliberately	30
16	Evaluation: making practical coverage meaningful	30
16.1	Separate tasks that look superficially similar	30
16.2	Coverage matrix	31
16.3	Benchmark provenance and held-out evaluation	31
16.4	Metrics beyond solved count	31
17	Design alternatives and final recommendation	32
A	Repository source map and version scope	32
B	Artifact guide and reproduction	33
C	The minimal continuation mechanism	34
	References	35

1 The design decision

The right successor is not “Djex translated into Lean.” It is a native dependent-term synthesizer that retains the useful lessons of Djex while removing the requirement that Lean types first fit a Haskell-oriented search language. Its central unit of work is a contextual judgment, not a string, a simple type, or an independently solved hole:

$$\mathcal{E}; \Gamma \vdash e : \mathsf{T}, \quad \mathcal{E}; \Gamma \vdash p : \mathcal{C}(e).$$

The environment $\mathcal{E}$ contains imported declarations; Γ is an ordered local telescope, including types, terms, propositions, let-bindings, and local instances; T is the exact requested type; and $\mathcal{C}$ is an optional behavioral proposition about the generated term. A successful constrained result contains both e and p . A useful unconstrained result contains e and a type-checking receipt, but it makes no unstated claim about intended behavior.

This choice has three important consequences. First, dependencies between types and terms are useful information rather than obstacles to be erased. A goal such as `Vec alpha (n + 1)` should expose an index constraint, not become an opaque atom unless a particular accelerator deliberately abstracts it. Second, proof search and program search must share choices. A proof failure about an earlier program fragment must be able to reopen that fragment. Third, the trusted acceptance boundary is deliberately smaller than the search implementation: a complicated heuristic is allowed to make poor suggestions, but not to redefine success.

1.1 What the present repositories already provide

The baseline is the Leant tree at commit `6bf05ad78c467989e68290f2d08bbbed40802d485`, with Djex submodule `e237e8667190aff38faaa590ce5b12afbfffac452`. The reviewed files are identified in Appendix A. This is a source-grounded design review, not a rerun of the repositories’ full test suites.

Djex already separates a shared synthesis foundation from Djinn’s logical search and Exference’s ranked search. Its architecture treats checked inventories, source identities, and candidate evidence as owned values rather than informal annotations [1]. Leant’s fragment translator documents the remaining semantic gap: some type applications retain structure, but term-indexed applications and other unsupported dependent subterms can remain opaque; negative claims are correspondingly restricted [2]. The behavioral checker already distinguishes a proof of the requested proposition, a proof of its negation, and an inconclusive attempt. It also retains the actual candidate when checking a contract that might ignore it [3].

Therefore, the rewrite must not regress to a simplistic “enumerate strings and run Lean” architecture. Nor should it advertise rank-N support, exact dictionaries, or candidate replay as entirely new inventions. The improvement sought here is a broader native search space and a better integration of dependency, behavior, recursion, and resources.

The existing Lean rewrite analysis already advocates exact expressions and a project-matched worker boundary [4]. The present article takes those principles as a starting point and specifies the missing operational design: how to construct dependent terms, coordinate shared holes, generate recursive programs, propagate contracts, and distinguish empirical success from justified completeness.

1.2 A practical scope, not a percentage promise

“A large fraction of practical Lean types” is an evaluation goal, not a theorem. It depends on the user population, available libraries, admissible axioms, whether library reuse counts, whether a useful

specification is supplied, and the resource budget. A type like `List alpha -> List alpha` has a trivial inhabitant; synthesizing the intended stable sort is a different task. A type involving an inaccessible noncomputable object may have a mathematical inhabitant but no admissible executable result.

The initial product should target polymorphic plumbing, products and records, user-defined inductives, indexed constructors, dependent adapters, short library compositions, structurally recursive traversals, and programs with tractable equational contracts. Stronger mathematical proofs, invariant discovery, mutually recursive indexed families, quotient computation, and effects belong to explicit capability tiers, not a catch-all success claim.

Recommended identity. Leant2 is a native, proof-carrying program synthesizer with a portfolio of bounded search methods. It is neither a universal inhabitation decider nor merely a new name for an existing proof tactic.

2 The query and result contracts

2.1 Freeze the question before searching

A query is the tuple

$$Q = (\mathcal{E}, U, \Gamma, \mathsf{T}, \mathcal{C}, X, \mathcal{G}, \Pi, \mathcal{B}).$$

Here U is the universe-parameter context, X is a finite set of typed observations, $\mathcal{G}$ is the admitted construction grammar and provider inventory, Π is the trust and executability policy, and $\mathcal{B}$ is the resource policy. These components are elaborated and sealed before speculative search begins.

The statement cannot contain an unsolved elaborator metavariable that search is free to instantiate to make the problem easier. Query elaboration should resolve synthetic metavariables and report unresolved input ambiguity before freezing. Universally quantified universe parameters remain parameters. A user may explicitly ask the synthesizer to choose a type or an index, but that is represented as an owned synthesis hole under a stated binder, not as an accidental ambiguity in the specification.

Consider a user request with an overloaded operation. Its selected instance is part of the elaborated question. A solver must not silently change the instance and then prove a proposition about a different operation. Similarly, a local assumption appearing in the proof remains a parameter of the exported declaration. It does not disappear just because it was implicit in the editor context.

For $\mathsf{T} : \text{Type}$, the familiar encoding

$$\{f : \mathsf{T} \mid \mathcal{C}(f)\}$$

is convenient and is used in several experiments. It is not the only internal encoding. Since queries can target arbitrary `Sort u`, the interface should store a program root and a proof root separately, with a common context. A sort-polymorphic wrapper such as an appropriately formed dependent pair is an implementation option, not a reason to restrict all targets to the universe accepted by a particular wrapper.

2.2 Behavioral inputs are Lean propositions

The primary constraint language is Lean itself. For example, the intended proposed frontend could offer:

```
-- Proposed surface syntax, not implemented by the prototype.
synth successor : Nat -> Nat
  satisfying fun f => forall n, f n = n + 1
  using [Nat.succ]

synth mapVec : (alpha -> beta) -> Vec alpha n -> Vec beta n
  satisfying fun g => forall f xs,
    toList (g f xs) = List.map f (toList xs)
```

The frontend elaborates the target and predicate using the project’s namespaces, notation, coercions, and instances. Specialized arithmetic or finite-domain constraints are derived views of the resulting proposition. They do not constitute a second public language whose semantics can drift from Lean.

Examples are equally explicit. A list of observations $f(a_i) = b_i$ is a finite conjunction, not a disguised universal specification. A property-based test plan is a search aid. A universal contract remains universal even when the current test bank is empty. If the full contract is too difficult to prove, the result can be presented as test-consistent only in a mode that permits that status; it cannot be silently accepted as certified.

2.3 Statuses that cannot be conflated

Status	Exact meaning
Typed candidate	A term checks at the frozen target. No additional behavioral claim is made.
Test-consistent candidate	A typed term passed the stated finite observations or tests. This is not universal correctness.
Certified candidate	The term and a proof of the exact requested contract pass acceptance under the stated assumptions and policy.
Refuted candidate	A checked counterexample or proof of the negation rejects this candidate. It says nothing about all other candidates.
Grammar exhausted	A specified finite search space was completely examined, with enough evidence to justify that particular exhaustion claim.
Search exhausted	A configured resource limit was reached or available search lanes ended without the necessary completeness evidence.
Unsupported	An essential requested capability or admissible representation is outside the enabled implementation.
Exact negative result	A Lean proof of the appropriate negative statement, or a separately described theorem about an exact formal fragment, has been obtained.
Infrastructure failure	Worker crash, canceled request, malformed solver response, or environmental inconsistency. Not mathematical evidence.

A candidate can be typed and refuted, or typed and still awaiting verification. These are best represented as orthogonal evidence fields internally, rather than a single Boolean. The table describes user-visible summaries.

A “best” result is also qualified. It means best among the verified candidates actually considered under a declared cost policy. Global minimality requires exhaustive exploration below the candidate’s cost and an appropriate lower-bound argument. The ordinary interactive mode should not imply it.

2.4 Trust profiles and computational requirements

The trust policy has separate fields for admitted logical assumptions and operational requirements. A strict constructive profile can forbid all additional axioms. An ordinary mathematical profile may admit Lean’s standard logical axioms while still excluding `sorryAx`, arbitrary new axioms, and unauthorized premises. An executable profile additionally requires that the selected program compile and respect an explicit effects policy.

These dimensions must not be collapsed. A classical proof about an executable function need not make the function noncomputable: the proof can be erased. Conversely, an axiom-free term built using a recursor need not be accepted by a particular code-generation route. The vector experiment in Section 13 encountered exactly this second issue. Checking the program’s dependency closure and its actual compilation route is different from merely inspecting the proof’s axiom list.

3 System architecture and semantic ownership

3.1 A Lean controller and project-matched workers

The frontend should be a Lean library exposing tactic, term-elaborator, and command interfaces, plus an optional Lean executable for REPL and batch use. Each project-local worker is built with that project’s toolchain and imports. A lightweight controller manages requests, cancellation, progress, and worker replacement. There is no Haskell frontend or Haskell semantic compatibility layer.

Within a worker, synthesis and ordinary Lean metaprogramming can share an environment snapshot. Process isolation is still valuable for untrusted plugins, external solver calls, memory exhaustion, and irreversible effects. An in-process proof tactic is the fast path; a replaceable worker is the robustness boundary. The architecture does not require the worker to exchange Lean internal pointers with a different toolchain.

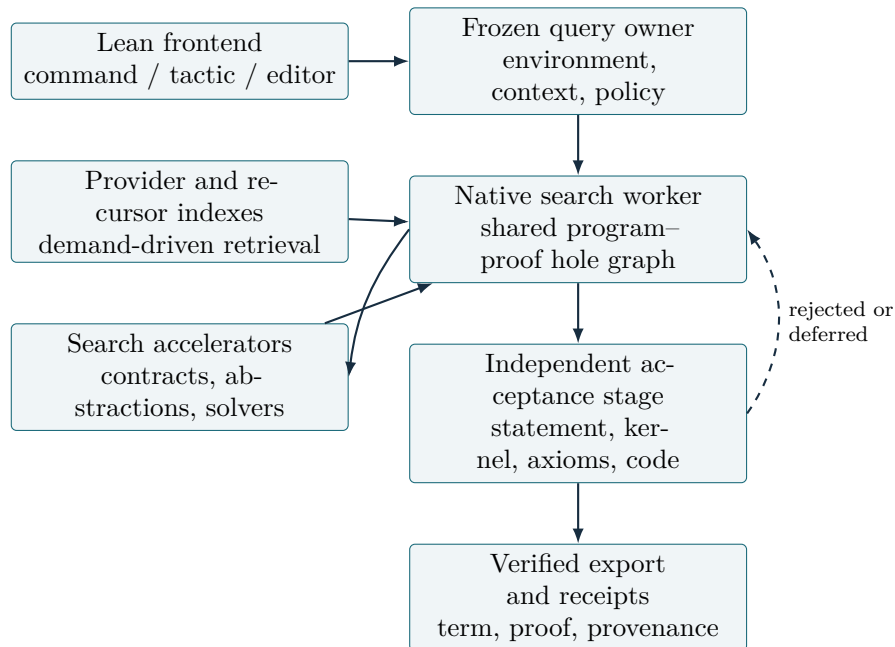


Figure 1: Proposed architecture. Search accelerators suggest and filter under explicit contracts; only the acceptance stage can issue a certified result.

3.2 One semantic owner, several derived views

The query owner stores the authoritative elaborated target and contract, their telescope, the admitted environment, and policy. Indexes, feature vectors, abstraction graphs, pretty-printed text, and test signatures are derived views. None may replace the original values at an acceptance boundary.

Lean’s `Expr` is a syntax representation, not an intrinsic proof that the represented expression is well typed. The advantage of using it is exact vocabulary and access to Lean’s own operations, not permission to omit validation. An application assembled with `mkAppN`, for example, can still be ill typed. A local free-variable identifier is meaningful only relative to its local context. A metavariable identifier is meaningful only relative to its owning metavariable context.

The proposed internal ownership hierarchy is:

```

QueryOwner
  frozen environment / universe parameters / local telescope
  exact target / exact contract / admissibility policy
  provider inventory version / grammar description

BranchOwner
  query reference
  branch-local Meta state and local-context views
  program root / contract root / dependency graph
  pending constraints / continuations / search provenance

AcceptedCandidate
  query identity
  closed program / closed contract proof
  acceptance receipt / axiom and computation reports

```

This is a schematic interface, not a compilable declaration. In particular, an accepted candidate should be constructible only by the verifier. A public record whose fields users can fill arbitrarily is not a meaningful authority boundary by itself.

3.3 Persist plans, not dangling metavariables

There are two useful persistence formats. In a live worker, persistent Lean state snapshots support cheap branch rollback. Across processes or sessions, save a replayable construction plan containing declaration names, universe applications, binder positions, rule choices, and explicit witnesses, or a validated closed expression in a pinned environment. Do not serialize a raw `MVarId` and assume it denotes the same unknown after reloading.

A construction-plan format is not a new approximate type system. Its purpose is transport and replay. All types referenced by the plan are reconstituted as Lean expressions and checked in the destination worker. Unknown or incompatible environment identities cause replay failure, never reinterpretation. The displayed source is generated only after a candidate has passed semantic checks, and its copy-paste replay is a separate usability check.

3.4 Acceptance is a transaction

The acceptance stage closes the candidate under the frozen telescope, instantiates only legitimately solved synthesis metavariables, and checks that no unresolved expression or universe metavariables

remain. It then validates the program declaration against the frozen target and the proof declaration against the frozen predicate applied to that program. Both declarations are retained even when the predicate ignores its argument.

The transaction runs in a scratch environment with kernel checking enabled. It rejects unexpected environment mutations and unauthorized newly introduced assumptions. It audits dependencies of both the program and proof, not just the final printed theorem. Where execution is requested, it compiles the actual program and records the result separately. The user’s project receives no declaration until the entire transaction succeeds.

For a higher-assurance deployment, candidate-producing metaprograms are treated as untrusted code and the exported declarations are replayed in a separate controlled checker process against a separately prepared challenge. This strengthens the ordinary tactic/kernel boundary, which by itself does not sandbox arbitrary metaprogram effects. Lean’s validation documentation distinguishes such assurance levels [5]. The experiments here used remote compilation and explicit axiom audits, not that stronger independent-checker workflow.

4 The native search state

4.1 A dependent AND–OR graph

A partial candidate consists of a root expression with holes and a graph recording how those holes were introduced. Each hole h has a local telescope Γ_h , a target T_h , a role, and a set of unresolved dependencies. Roles include program term, type argument, universe choice, instance argument, contract proof, termination proof, and invariant.

An application of a function can create an AND node: all required arguments must be constructed. A choice of constructor or provider is an OR node. Unlike a propositional proof tree, however, the child problems are not generally independent. The type of a later argument can mention an earlier argument; an instance can determine a computational operation; and a contract can reject an otherwise well-typed program choice.

For the target

$$\{n : \mathbb{N} \mid n = 1\},$$

a constructor choice produces a term hole $h : \mathbb{N}$ and a proof hole $q : h = 1$. The assignment $h := 0$ solves the first hole but blocks the second. Treating the first solution as permanent loses the valid result $h := 1$. A product of independent “first-answer” solvers is therefore not a correct search architecture for the desired behavior.

4.2 Continuation-aware backtracking

The basic control rule is: a choice succeeds only when its newly created obligations *and the surrounding continuation* succeed. A small depth-first version is:

```
solve(fuel, pendingGoals):
  if pendingGoals is empty: return success
  if fuel is zero: return unknown(limit)
  choose an eligible goal g
  save the complete allowed branch state S
  for action in alternatives(g):
    restore S
```

```

outcome := run action with its own resource limit
if outcome introduces children:
  result := solve(fuel - 1, children ++ remainingGoals)
  if result is success: return success
  retain diagnostic or deferred work as appropriate
restore S
return no-success-at-this-budget

```

This is the mechanism tested by the native prototype. Production search should expose resumable alternatives instead of relying on the host call stack, but it must preserve the same dependency semantics. Merely running a tactic separately on every hole and concatenating the results is not equivalent.

The state restored for a speculative rule includes all mutable information the rule is allowed to change: metavariable assignments, local-instance state, postponed constraints, and any owned elaborator state. A strictly **MetaM**-only rule can have a narrower transaction than a tactic invoking term elaboration. Commands that create declarations or external effects do not belong inside a rollback-only Meta transaction; they require an isolated scratch environment or subprocess. Names allocated during unsuccessful branches are not reused as if they identified the same objects.

4.3 Which goal should be solved next?

The default choice is the most constrained eligible goal, subject to dependency barriers. A constructor-index equation or a determined instance obligation can cheaply constrain several program holes. A proof goal involving an unknown function may instead be deliberately delayed until enough program structure exists. The scheduler computes watchers from each unresolved metavariable to the goals whose targets or constraints mention it.

When an assignment occurs, only affected goals are reclassified. A flex-rigid equation that becomes pattern-like is retried. A class obligation whose type parameters become known is retried. A contract whose application now reduces is simplified. This event-driven scheme avoids repeatedly normalizing every goal after every small step.

Syntactic separation is not enough to prove independence: the absence of the same hole identifier from two printed targets does not rule out shared universe or local-instance constraints. An implementation may optimize genuinely independent subgraphs only after computing their full dependency support.

4.4 Freezing and reopening program choices

For efficiency, a candidate sent to expensive verification is frozen as a closed term relative to the query telescope. The verifier may not solve its proof obligation by changing the candidate. A successful proof is evidence about that frozen candidate. A failed proof returns either a real counterexample, a scoped search conflict, or an unknown result.

The search engine can then reopen an earlier program choice in its own branch graph. This does not contradict freezing: the replacement is a new candidate with a new identity. The proof of the previous candidate cannot be attached to it without an explicit equality transport or a fresh proof. This rule prevents a subtle form of evidence reuse in which a shared metavariable assignment silently changes the subject of an already accepted proof.

5 The provider inventory and type-level search

5.1 Retrieve a bounded relevant inventory

A production project may expose a very large environment. Exhaustively trying every constant at every hole is neither a useful ranking policy nor a sensible first implementation. Build persistent indexes when modules are imported, keyed by weak-head result heads, argument/result dependencies, inductive family, names, and optional user tags.

The default retrieval sequence is local variables and local instances; explicitly requested providers; constructors and projections relevant to the target; tagged library combinators; a namespace or module neighborhood; and finally a broader environment fallback. Each expansion is visible in the query’s effective grammar and budget record. A user can lock the inventory for reproducible searches.

An index must include wildcard and reducible-head buckets. If a result head is a variable, or is revealed only after reduction, excluding it from all concrete-head buckets can lose a useful provider. The index is a retrieval accelerator, not a proof of impossibility. Broader retrieval can be resumed when a narrow inventory is exhausted.

5.2 Instantiate full dependent spines

Given a provider

$$c : \Pi(x_1 : A_1) \cdots (x_k : A_k(x_1, \dots, x_{k-1})).R(x_1, \dots, x_k),$$

create fresh universe metavariables for this occurrence of a global constant and open its full telescope in order. Explicit, implicit, strict-implicit, and instance-implicit binders are retained. Later argument types are instantiated with the actual earlier arguments or their shared holes. Unify the resulting output with the target under branch-local Lean definitional equality.

Some explicit arguments can be inferred from result matching; some implicit ones remain genuine search problems. The binder’s presentation category is not a reliable criterion for whether it needs search. In particular, automatically filling every implicit argument with an unconstrained metavariable and then forgetting it is not a solution.

Local providers are different from global schemes. Their free-variable identities and existing universe parameters must be preserved exactly. Freshening a local provider as though it were a global polymorphic declaration can turn a fixed universe or a fixed dictionary into a new choice.

5.3 Type arguments are terms too

Moving to Lean removes an artificial ceiling on the forms of type expression retained in the search state. It does not make type-level synthesis decidable. When an argument has type `Sort u`, demand-directed alternatives should first come from types visible in the target, local hypotheses, instantiated provider parameters, and declared input/output relationships. Only then should a bounded type-expression grammar construct products, sums, dependent pairs, function types, and type-constructor applications.

This grammar is scoped. A type proposed under a binder may mention that binder, but may not escape it. Its universe constraints must be solved by Lean rather than flattened to a single “proper type” category. Lean’s universe hierarchy remains part of the problem; the impredicativity of propositions does not make arbitrary **Type**-level instantiation universe-free [6].

For unresolved higher-order equations, try Lean’s ordinary elaboration and unification machinery under bounded transparency, then explicit pattern-oriented or demand-generated alternatives. A stuck

equation is not necessarily inconsistent. It can be postponed, escalated to a different transparency level, or left unsupported by the current strategy. The general higher-order problem is not solved merely by calling `isDefEq`.

5.4 Dictionary evidence has computational identity

Two values of the same class type can implement different operations. Therefore, cache keys and candidate identity cannot erase dictionary terms just because their types agree. In ordinary Lean-source mode, instance search should follow Lean’s usual local and global resolution policy. In an explicitly selected-provider mode, the user may supply a particular dictionary as an ordinary term; the generated application must retain it.

The native prototype exercises this boundary modestly through `Inhabited.default` and a local instance. It also includes two same-typed payload records where the contract selects the right-hand value. It does not implement a general alternative-instance exploration policy. Such a policy should be opt-in and should show the selected instances in the generated result, rather than silently searching for an unusual interpretation of overloaded notation.

6 Focused native construction

6.1 Rules, priorities, and side conditions

The core search engine is bidirectional. It constructs introduction forms when the target determines useful structure, and synthesizes elimination forms from local or retrieved providers. Every rule returns a collection of new obligations together with a continuation that rebuilds the parent expression. The following are the main rules; the word “try” always means a branch-local, bounded transaction.

Rule	Construction	Required discipline
Pi introduction	For target $\Pi x : A.B$, introduce $x : A$ and solve B .	Preserve binder information, dependencies, and universe parameters.
Exact local	Use a local term whose inferred type is definitionally equal to the target.	Equality checks can assign only branch-owned unknowns.
Provider application	Open a full dependent application spine and constrain its result by the target.	Retain every argument and all shared constraints.
Constructor	Read constructors from the target’s inductive metadata and try compatible result indices.	Constructor and field choices remain backtrackable.
Projection	Apply a field projection to a known structure value.	Dependent field types are instantiated with that exact value.
Reflexivity	Close an equality whose sides are definitionally equal.	Freeze candidate ownership before issuing proof evidence.
Transport	Use a known equality to move a term or goal between propositionally equal types.	Build a checked transport; do not identify non-definitional types.
Case split	Eliminate a selected local inductive value using an appropriate motive.	Respect dependent locals, index equalities, and elimination restrictions.

Rule	Construction	Required discipline
Recursion sketch	Instantiate a registered structural or well-founded template.	Every recursive call has a justified smaller argument or decrease proof.
Proof backend	Run a bounded proof-producing tactic on a proof obligation.	A closed result is required; partial progress is not certification.

Lambda introduction is the default for a determined function target, because it exposes the arguments needed to construct a body. This is a focusing policy, not a license to mark all introduction rules irreversible. Dependent-pair witnesses, disjunction alternatives, and data constructors make semantic choices. Even constructor fields that look independent may later occur in the contract.

A completeness-oriented lane can use an explicitly defined long-form grammar in which function introductions are mandatory. A production lane may additionally try direct provider reuse before eta expansion to obtain smaller source terms. The guarantee belongs to the declared grammar and normalization assumptions, not to the heuristic label “safe.”

6.2 Normalization is controlled work

Weak-head normalization usually reveals whether a target is a function, a reducible alias, or an inductive family. It should be memoized within the relevant environment and assignment version. Deeper simplification is demand-driven: unfold the head needed for a rule; simplify an index equation; reduce a closed sample; or expose a contract’s outer connective.

Unfolding every definition at every node destroys abstraction and can make large proof terms or recursive reductions dominate search. Conversely, never unfolding loses straightforward adapters between definitionally equal types. Use a small ordered transparency ladder and a per-operation budget. A failure at one transparency level can schedule an escalation; it is not a permanent incompatibility certificate.

The implementation must charge normalization, unification, and elaboration work to the request even if the branch is later rolled back. Restoring a metavariable context is not a refund of computational work. This is particularly important when a malicious or simply unfortunate goal repeatedly triggers expensive reduction before failing.

6.3 Application search without premature argument choices

For a candidate provider, result-directed matching often determines some type arguments and indices before any value search is necessary. Build the full spine, constrain the result, and classify the resulting holes. Resolve arguments whose values are uniquely determined by unification; attempt local instance resolution when its parameters are ready; then schedule remaining computational arguments and proof obligations according to dependencies.

For example, a provider

$$\text{get} : \Pi(n : \mathbb{N}). \text{Vec } A \ n \rightarrow \text{Fin } n \rightarrow A$$

should share the same n between the vector and bounded index. Selecting a vector at length m must constrain the index to $\text{Fin } m$. A backend that separately synthesizes “some vector” and “some bounded natural” and then asks the renderer to repair the mismatch has discarded the principal benefit of native search.

If application creates a proof precondition, it becomes an ordinary proof hole linked to the arguments that determine it. Arguments whose proof obligations are difficult can be deprioritized; they cannot be accepted just because their erased runtime behavior looks plausible.

6.4 When proof automation helps and when it does not

Use existing proof-producing automation for proof obligations instead of rebuilding every arithmetic or logical decision procedure inside Leant2. A first portfolio can include core simplification, reflexivity, constructor reasoning, **decide**, arithmetic normalization, and project-enabled tactics such as **aesop**, **omega**, and **grind**. Aesop’s documented rule-based search is a natural component, while Lean’s current **grind** interface provides another proof-search route [8, 9].

These backends do not replace the outer program search. A theorem prover may solve a contract after the candidate is fixed, but it usually does not expose the precise program alternatives the user wants ranked. Its successful proof is a returned artifact; its failure merely means that backend did not close the goal under its budget. In particular, logical proof irrelevance does not justify erasing the computational choices that determine the proposition being proved.

A backend adapter has the contract

$$(\Gamma, P, b) \mapsto \text{Proved}(p) \mid \text{Refuted}(q) \mid \text{Unknown}(r),$$

where $p : P$ and $q : P \rightarrow \text{False}$ are checked in the same frozen context. Most tactics implement only the first and third outcomes. Progress states can be retained for subsequent search, but they do not satisfy **Proved**.

7 Dependent elimination and indexed data

7.1 Construct the motive from the whole telescope

Suppose a local value $x : I \vec{p} \vec{i}$ occurs in the target and in the types of later locals. A case-split rule cannot merely replace x by each constructor in the printed goal. It must preserve the dependent telescope and derive a legal elimination motive.

The proposed algorithm first computes the transitive dependency closure of x : later local declarations whose types or values depend on x , on its indices, or on already selected dependent locals. It generalizes the needed declarations into the motive, retaining their order. It then invokes Lean’s dependent elimination machinery in a sandboxed state, rather than inventing constructor equations by textual substitution. The generated branch contexts are read back as exact local contexts. Constructor index constraints are solved by definitional equality or by explicit equality reasoning, and branch programs are synthesized under those refined contexts.

Finally, the parent term is rebuilt through the checked eliminator or an elaborated pattern-match plan. If generalization produces an excessively complex motive, the rule may decline or enumerate a bounded alternative. Incorrectly generalizing less is not an admissible shortcut.

7.2 Impossible branches require evidence

An indexed constructor can be inapplicable because its result indices cannot equal the scrutinee’s indices. For a vector known to have successor length, the nil branch is impossible. The elimination

machinery may discharge it using equality or no-confusion principles. A failed unification attempt by a heuristic is not, by itself, a proof that the branch is impossible.

This distinction matters especially when indices are symbolic expressions. Two index expressions may be propositionally equal even when reduction alone does not identify them. Conversely, a solver’s unsatisfiable answer about an approximation of the indices is not sufficient unless its encoding and reconstruction are justified. Leant2 should record whether each eliminated branch was removed by a kernel-checked contradiction, by a trusted exact fragment theorem, or merely omitted by a heuristic lane. Only the first two can support an exact coverage claim.

7.3 Equality transport is an explicit search action

If $h : a = b$ and $x : \beta(a)$ are available, a term of $\beta(b)$ is constructed by equality elimination. The target is not necessarily definitionally equal to $\beta(a)$. A transport rule can try relevant equalities in either orientation, with a small cost that discourages unnecessary casts.

A production implementation should maintain a proof-producing equality graph for relevant indices and dependent arguments. Paths carry equality proofs, not just union-find representatives. When a path is used to change a type, the corresponding proof is inserted into a checked transport. Congruence closure may help discover a path, but dependent congruence sometimes requires a heterogeneous equality or an additional transport. Restrict the first implementation to cases it can reconstruct explicitly, and return an unknown result outside them.

Rewriting a term for presentation is also different from proving it equal to another term. Only definitional equality, or an explicit equality proof with a correct transport of the contract, justifies reusing evidence across the rewrite.

7.4 Elimination restrictions are part of the language

A native synthesizer must distinguish propositions from computational data. An existential proof in `Prop` is not generally a permission to extract its witness into arbitrary data. Likewise, `Nonempty alpha` is not interchangeable with a supplied value of `alpha` in an executable constructive search. Any use of classical choice must be explicit in the policy and reflected in the dependency report.

Quotients present a different boundary. A quotient-lift construction requires proof that the proposed function respects the quotient relation. It is not ordinary pattern matching on a hidden representative. The appropriate Leant2 plugin therefore creates both the function-body holes and the compatibility-proof hole, retaining their dependency. Indexed inductives, proposition elimination, quotient lifting, and universe-polymorphic elimination should have distinct acceptance tests rather than being grouped under a vague “dependent types supported” label [7].

7.5 Worked indexed-constructor trace

For

$$A \rightarrow \text{Vec } A \ n \rightarrow \text{Vec } A \ (n + 1),$$

Pi introduction exposes $a : A$ and $xs : \text{Vec } A \ n$. The result-head index retrieves the vector constructors. Trying `nil` fails to produce a compatible result. Trying `cons` creates a head hole of type A and a tail hole of type $\text{Vec } A \ n$, with the constructor’s implicit index tied to the target. Local reuse solves them with a and xs .

For the more interesting result $\text{Vec } B \ n$ from a function $A \rightarrow B$ and an input vector, constructors alone are insufficient when n is abstract. A case or recursive template exposes the index. In the nil branch, n refines to zero and nil is available. In the cons branch, n refines to a successor and the smaller recursive result supplies the tail. This is the intended integration between dependent elimination and structural synthesis.

8 Recursive program synthesis

8.1 A ladder of recursion methods

The safest and most efficient first attempt is library reuse: map, fold, traversals, recursors, and specialized verified combinators. If no suitable composition is found, synthesize a registered structural template. Only later try generalized or well-founded recursion. The search should avoid admitting the desired function as an unrestricted provider of its own type; that produces circular terms rather than terminating programs.

A recursion descriptor records the function telescope, selected decreasing arguments, admitted recursive-call forms, result motive, generalized parameters, and the termination evidence needed by each call. Each occurrence of a recursive call is a restricted capability: it can be applied only with arguments whose decrease or structural-subterm relation is justified in that branch.

For the initial list descriptor, the calls admitted under a cons branch are calls on the tail, with unchanged fixed parameters and explicitly generalized accumulator parameters. For trees, calls are on immediate recursive fields. For vectors, the recursive field carries both a smaller structure and its refined length index. Mutual inductives require a descriptor covering the entire mutually recursive family, not independent invented calls between arbitrary result types.

8.2 Generate programs and induction obligations together

For a template with clauses $e_1, \dots, e_k$, the system creates computational holes for the clause bodies and proof obligations for their contracts. The induction hypothesis is available only for the recursively defined smaller computations licensed by the template. It is not an axiom asserting the full contract for every input.

A useful implementation is a two-pass checked template. First construct a structurally recursive function with explicit body holes. After those holes are completed and the function passes Lean’s termination and compilation checks, generate a theorem with the matching induction principle. Clause-specific summary constraints can guide the first pass, but the completed second pass proves the original contract. An alternative interleaved implementation is possible, provided the logical induction hypotheses and computational recursive calls remain correctly scoped and the final declarations are closed and checked.

The prototype uses a deliberately narrower form of this method: the vector pattern-match skeleton and smaller recursive call are supplied by hand, while the branch bodies are synthesized. Its behavior theorem is also supplied and checked. It is evidence for the native branch interface, not evidence that automatic recursion-scheme discovery has been implemented.

8.3 Infer a useful carrier before enumerating a large body

Many apparent recursion problems are failures to choose a rich enough result carrier. A fold computing a left-associated accumulation can use a function-valued carrier:

$$\text{foldr } (\lambda x \ k \ a. \ k(\text{step } a \ x)) \ (\lambda a.a) \ xs \ a_0.$$

Here the recursive fold result has type $S \rightarrow S$, not merely S . This carries the continuation needed to process the remaining accumulator.

Leant2 should infer candidate carriers from the result telescope and recursive demand. Start with the direct result type R . If a changing argument $a : S$ appears in the desired recursive equation, try $S \rightarrow R$. If two mutually needed summaries occur in the postcondition, try a product carrier. If later fields depend on earlier summary values, try a dependent pair. Every constructed carrier is subject to the normal universe and type checks, and its complexity is charged to the grammar budget.

This carrier grammar is intentionally finite at each budget. It does not enumerate arbitrary inductive definitions. A failed simple carrier can trigger a bounded generalization rather than merely increasing the number of candidate terms in the same inadequate grammar.

8.4 Accumulator generalization and strengthened contracts

Consider synthesizing a tail-recursive reverse. The desired public contract may be

$$f(xs) = \text{reverse}(xs).$$

A candidate auxiliary function takes an accumulator. Its useful strengthened contract is

$$g(xs, acc) = \text{reverse}(xs) ++ acc.$$

The recursive call uses $g(xs, x :: acc)$, so the induction hypothesis must quantify over the accumulator rather than fixing its original value. A tactic that inducts before generalizing acc often obtains an unusable hypothesis.

The proposed strengthening algorithm inspects the template's recursive-call substitution. Parameters changed by that substitution are candidates for generalization. A finite grammar of relational summaries is built from the public contract, constructor equations, and tagged library lemmas. Candidate strengthened contracts must imply the public one at the initial state, and their base and step preservation obligations are checked. The system never assumes a strengthened invariant merely because it fits observed examples.

This can be expressed as a small constraint system over summary holes. For a recursive strongly connected component, the summaries are checked simultaneously with a well-founded induction argument. Proving each function using an unchecked summary of the other is circular and prohibited.

8.5 Well-founded recursion as an explicit later capability

Some useful algorithms recur on values that are not syntactic subterms: Euclidean-style reductions, divide-and-conquer partitions, or work-list processes. A well-founded descriptor includes a relation $\prec$, a proof of well-foundedness, and obligations showing that each recursive argument decreases. The first search space can use user-supplied measures, natural-valued measures extracted from lengths or sizes, and lexicographic tuples of such measures.

The measure grammar and arithmetic solver are bounded. A decrease proof that times out suspends or rejects that sketch at the current budget; it does not imply the program is mathematically nonterminating. Conversely, adding `partial` is not a repair for a failed termination proof when the request asks for a total implementation. General unrestricted recursion and effects are outside the default total synthesis grammar.

8.6 Compile the form that will actually be exported

Direct recursor applications are excellent semantic building blocks but are not universally interchangeable with the compiler’s preferred recursive source forms. The experiment’s first vector implementation produced an axiom-free recursor term but was rejected by the code generator. Re-expressing the implementation as a structural pattern match with the same intended smaller call succeeded.

Accordingly, a recursive plan has both a semantic construction and an export/compilation strategy. The latter must be tested, not assumed from the former. A proof-only request may legitimately stop after logical validation; an executable request must pass the actual code-generation stage without silently switching to noncomputable mode.

9 Contract-directed search and semantic domains

9.1 Search with the specification, not only after it

A generate-and-test architecture discards valuable information. If the contract says that a result has length $n + 1$, it should influence constructor selection before a complete program has been enumerated. If the contract specifies a sum, it should constrain fold coefficients or recursive summaries before searching millions of arithmetic expressions.

The native design supports both a generic route and specialized domains. The generic route constructs $e : T$ and seeks a proof of $\mathcal{C}(e)$ in the shared hole graph. A domain plugin recognizes a fragment of the exact contract, produces a simpler constraint problem, and returns candidate terms together with proof obligations or certificates. Failure to recognize a domain leaves the original query intact.

Refinement-type synthesis demonstrates the value of decomposing a specification during construction [10]. Leant2 should borrow that operational idea without replacing arbitrary Lean predicates by a globally assumed decidable refinement language. Decidability and completeness belong to each admitted domain.

9.2 A domain adapter has four proof-relevant boundaries

A semantic domain needs a typed interpretation of candidate syntax, a meaning for abstract states or summaries, justified transfer rules, and a reconstruction path back to the requested Lean proposition. These are distinct responsibilities.

For example, a length domain can attach a summary $\ell_{out} = \ell_1 + \ell_2$ to append. It may use that summary to select a provider composition. The final acceptance still needs a Lean proof about the generated function’s actual list length. If a summary describes only an upper bound, it cannot be used as an exact equality. If an abstract interpreter omits an unsupported operation, the omission must be modeled as unknown, not as zero cost or zero length.

The essential direction for impossibility pruning is overapproximation: every concrete behavior under consideration must be represented abstractly. Then a genuinely impossible abstract constraint can exclude the corresponding concrete candidates. An underapproximation can find useful witnesses, but

failure inside it says nothing about unrepresented solutions. Each plugin declares which direction it supports and which conclusions it is allowed to produce.

9.3 Backward contract decomposition

For pure constructor terms, a goal predicate can often be reduced after substituting the constructor. A desired record invariant becomes constraints on its fields. For applications, tagged provider theorems act as compositional contracts. Suppose a provider has a checked theorem

$$\forall x. P(x) \rightarrow Q(c(x)).$$

To establish $Q(c(h))$, a plugin may generate the subobligation $P(h)$ and instantiate that theorem when the argument is complete. This is a sufficient backward rule. It need not characterize every way of proving $Q(c(h))$; an audit lane must retain a route that does not rely solely on an incomplete contract library.

For conditionals, split the proof under the branch assumption and its negation. For a recursive call, use only the appropriate induction hypothesis or verified recursive summary. For dependent constructor fields, later field contracts are instantiated with the actual earlier fields, not with independently abstracted placeholders that forget correlations.

9.4 Counterexample-guided refinement with three-valued verification

The main loop is:

```
CEGIS(query Q, candidate stream S):
  bank := typed examples from Q
  deferred := empty fair queue
  while budget remains:
    choose a new candidate from S or resume a deferred candidate
    freeze candidate e
    if a bank observation is proved false for e:
      reject e and retain the witnessed mismatch
      continue
    outcome := boundedVerify(Q.contract e)
    if outcome is Proved p:
      submit (e, p) to independent acceptance
    else if outcome supplies a replayed counterexample x:
      add x with its type, assumptions, and replay evidence to bank
      update affected candidate signatures
    else:
      defer (e, remaining proof strategies)
  report verified results and explicit unfinished work
```

The word “replayed” is crucial. A model returned by an external solver can be spurious because of an approximation, a sort mismatch, or different arithmetic semantics. It enters a proof-authorized rejection bank only after its meaning in Lean is established. A proof failure is not a counterexample, and lack of a counterexample is not a proof.

A domain can sometimes prove that a finite test family is complete for a declared grammar. Section 12 gives such an example. That is a theorem about the grammar, not a general conversion from testing into universal correctness.

9.5 External solvers are optional and untrusted

The initial Leant2 system can operate entirely within Lean for synthesis and proof reconstruction. An SMT solver can be added as a candidate generator, a parameter solver, or a counterexample finder. Its responses are data, not new axioms. A reconstruction layer translates a successful result into checked Lean evidence, or the result remains advisory.

The bridge must distinguish natural arithmetic from integer arithmetic, truncated subtraction from subtraction in a ring, fixed-width bitvectors from arbitrary naturals, partial operations from totalized Lean definitions, and solver equality from dependent equality. Quantifiers and uninterpreted functions require equally explicit interpretation. Unsupported constructs cause a domain refusal or a conservative unknown result.

The policy for native evaluation is also explicit. A fast evaluator can rank or test candidates, but a strict proof profile should not silently widen its trusted computing base to turn a runtime Boolean into a theorem. Use proof-producing reduction or an approved reflective checker for acceptance; report any stronger trust assumptions separately.

9.6 Observational equivalence is not semantic equivalence

For a test bank X , define the observational signature

$$\sigma_X(e) = (e(x))_{x \in X}$$

when those observations are meaningful and evaluable. Signatures are useful for diversity-aware ranking and caching repeated evaluations. They are not generally sufficient to merge candidates permanently. Two programs can agree on every current test and disagree on the next one. For higher-order programs, the set of arguments used by a surrounding context can also change after a substitution.

Therefore, finite signatures partition a priority queue, not the semantic candidate universe. Within a bin the system can keep a preferred representative for a bounded heuristic lane while retaining alternatives for later refinement or an audit lane. Permanent equality-based merging requires checked equality, definitional identity in the correct context, or an exact domain theorem that is strong enough for the intended continuation.

10 Fair search, abstraction refinement, and caching

10.1 Portfolio lanes with an explicit audit route

Useful synthesis methods have different strengths. A local focused lane finds short adapters quickly. A provider-composition lane searches libraries. A dependent-elimination lane explores cases and motives. A recursion lane instantiates structural templates. A semantic lane solves restricted contract constraints. A proof lane verifies frozen candidates. An optional logical-fragment lane can run a terminating propositional calculus on a precisely reified fragment.

The scheduler allocates bounded work slices among nonempty lanes and retains their continuations. No lane can consume the entire request indefinitely inside a supposedly single step: normalization, elaboration, proof tactics, and external calls have nested limits. A production request can stop early after a good certified result. A completeness-oriented audit request instead uses a fair cost-bounded enumeration and records every deliberate restriction.

Fairness does not mean equal wall-clock time per queue. A weighted round-robin schedule can give the likely productive lanes more work while ensuring that every enabled unfinished lane is revisited. Timeouts inside a branch produce resumable unknown work when appropriate. Cancellation stops the whole request and retires worker-owned resources; it is not interpreted as search exhaustion.

10.2 A finite grammar at every cost bound

Let each admitted construction have a positive cost, with explicit charges for applications, binders, constructor fields, recursive-template choices, type expressions, and literal encodings. Universe expressions and explicit instantiation choices also need bounded representations. A bound on the number of term nodes alone does not make the space finite if arbitrary numeric literals or arbitrary type expressions are available at unit cost.

One suitable audit grammar limits syntax size and literal bit length, uses a finite provider inventory, and bounds the depth and size of demand-generated types, motives, invariants, and measures. Proof terms can have a separate cost bound. Search increases these bounds by a fair diagonal schedule so that expensive proofs for small programs are not postponed forever behind larger programs with trivial proof obligations.

The production score may be

$$s(e) = w_1 \text{size}(e) + w_2 \text{casts}(e) + w_3 \text{recursionComplexity}(e) + w_4 \text{unusualProviders}(e),$$

with additional tie-breakers for proof size and source readability. This is a ranking heuristic, not automatically an admissible lower bound for optimal search. The interface should expose the policy name and permit deterministic replay of the same ordering.

10.3 Type-guided abstraction refinement as retrieval, not replacement

A provider-composition accelerator can construct a coarse graph of input and output type shapes. It searches that graph for a composition skeleton, attempts exact Lean instantiation, and refines the abstraction when the skeleton cannot be typed. Type-guided abstraction refinement has been developed for polymorphic component synthesis in TYGAR [11].

For Leant2, the abstraction must retain enough labels to diagnose why a proposed skeleton failed: nominal type heads, index dependencies, universe constraints, and dictionary identities. A conflict can introduce a new abstract distinction, for example separating vectors of zero and successor length or separating two nominal constructors previously merged. The exact branch state remains authoritative throughout.

This accelerator is particularly valuable for large provider inventories, but it should initially be a proposal mechanism only. Claiming that failure of its abstract graph excludes every Lean inhabitant would require a separate soundness and coverage theorem for the abstraction, grammar, and reconstruction route. Such a theorem is not supplied by the mere use of graph reachability.

10.4 Cache keys include semantic context

A reusable cache key includes the environment and inventory versions, transparency policy, canonical local telescope, local let values, selected instance evidence, target, relevant assignments, contract, and grammar policy. Canonicalization can rename bound variables, but it must preserve dependency order. Alpha-equivalence of displayed text is not enough to establish equivalence of contextual goals.

Positive cached terms are re-instantiated and checked in the destination context through an explicit context embedding. Negative entries carry their exact scope: a finite grammar and budget, a proved contradiction, or a heuristic failure. A timeout is never cached as permanent uninhabitability. New counterexamples invalidate observational bins or append to their signatures; a different contract gets a different verification identity.

Within one request, dependencies can support compact learned conflicts. If a branch fails because a specific constructor choice and a specific index assignment imply a checked contradiction, record that small assignment set. A proof tactic’s failure has no such logical status. It may justify deprioritization under the same tactic budget, but not a learned universal no-good clause.

10.5 Parallelism and determinism

Parallel workers share only immutable environment snapshots and validated cache entries. They do not share mutable metavariable contexts. A returned plan is replayed or a closed candidate is rechecked by the receiving owner. Final result ordering can be deterministic by cost and a stable candidate key even when completion order varies, or explicitly latency-first when immediate feedback matters more.

The benchmark harness should distinguish these modes. Comparing a latency-first parallel run with a deterministic serial run without recording their resource budgets makes performance and candidate-quality results difficult to interpret. The initial implementation should establish correct serial continuation semantics before adding parallelism; concurrency is an optimization of the search schedule, not a replacement for dependency tracking.

11 Guarantees and their exact assumptions

The following statements describe the proposed architecture. Their assumptions are part of the claims. In particular, sound result acceptance is much easier to obtain than completeness, useful candidate ranking, or good performance.

11.1 Acceptance soundness

Theorem 11.1 (Relative soundness of certified results). *Fix a query Q whose elaborated environment, telescope, target, and predicate are frozen. Assume the acceptance stage correctly closes the returned expressions under that telescope, checks them with a sound Lean kernel against the frozen declarations, and enforces the stated assumption policy. If it reports a certified result (e, p) , then, relative to the admitted environment and local assumptions,*

$$\mathcal{E}; \Gamma \vdash e : \mathsf{T} \quad \text{and} \quad \mathcal{E}; \Gamma \vdash p : \mathcal{C}(e).$$

The conclusion is independent of the correctness of the search heuristics.

Proof. The acceptance stage checks a declaration whose type is the closure of the frozen T and whose value is the closure of e . Kernel soundness gives the first judgment after reopening the telescope. It separately checks the closed proof against the frozen $\mathcal{C}$ applied to the same accepted program. Reopening gives the second judgment. Assumption auditing restricts the premises to those admitted by policy. The search algorithm occurs nowhere in this derivation: it supplies candidate syntax, which the acceptance stage checks rather than trusts. $\square$

The theorem does not prove that the formal contract expresses the user’s informal intention. It also does not prove that the runtime compiler preserves every semantic property, that external effects terminate, or that arbitrary metaprograms are safely sandboxed. These are separate obligations and are deliberately visible in the architecture.

11.2 Termination of a bounded audit run

Definition 11.2 (Finite effective search envelope). An envelope consists of a finite provider inventory; a finite term, type, literal, motive, measure, and proof grammar at each declared bound; a finite bound on rule depth or expanded states; and a terminating, resource-limited implementation of every primitive action. A timeout is an unknown result, not a successful decision.

Proposition 11.3. *A search confined to a finite effective envelope terminates, provided its scheduler does not reschedule the same unfinished action indefinitely without consuming a finite request resource.*

Proof. There are finitely many scheduled derivations or states under the envelope. Each primitive action returns within its limit. Every scheduled step either removes work, advances a bounded continuation, or consumes a finite request resource. Thus no infinite sequence of productive or repeatedly suspended steps is possible within the request. $\square$

This proposition says nothing about whether the budget is sufficient. In particular, a queue cap that discards candidates can preserve termination while invalidating an exhaustion or minimality claim. The discarded-work record must therefore be part of the audit result.

11.3 Relative completeness by dovetailing

Theorem 11.4 (Conditional relative completeness). *Fix a query, an admitted environment, and effective grammars for programs and contract proofs. Assume: (i) a pair (e, p) satisfying the query has a finite derivation in these grammars; (ii) each step of that derivation is eventually considered with sufficient primitive resource limits; (iii) scheduling fairly increases both construction and proof bounds; and (iv) no pruning rule removes every derivation of a valid pair unless justified by an equivalence preserving such a pair. Then the unbounded sequence of audit runs eventually returns a certified result, assuming the acceptance stage terminates on that pair.*

Proof. Choose a finite derivation of a valid pair. Its syntax sizes, rule count, provider selections, and the resources needed by each successful primitive computation have finite upper bounds. A fair diagonal schedule eventually reaches an envelope containing those bounds. Fair enumeration eventually visits the derivation or a validity-preserving replacement. By the pruning hypothesis, all such derivations have not been removed. The completed pair reaches the acceptance stage and is accepted. $\square$

This is not a completeness theorem for arbitrary Lean inhabitation. The selected grammar may omit the necessary recursor, type instantiation, induction invariant, equality transport, or proof. A finite practical request may stop long before the relevant envelope is reached. A production beam-search lane may intentionally violate assumption (iv). These limitations do not threaten Theorem 11.1; they limit discovery, not correctness of accepted results.

A subtle point is the proof grammar. Enumerating all program candidates fairly while giving each an eternally fixed proof timeout does not satisfy the theorem. A correct but hard-to-verify program can otherwise be starved forever. The same applies to a fixed unification timeout on a particular type instantiation.

11.4 Justified pruning

Proposition 11.5 (Contextual replacement). *Suppose e and e' are well typed in the same context and a checked proof establishes $e = e'$. If the contract proof is transported along that equality, replacing e by e' preserves existence of a certified result for that contract. Agreement on a finite sample set alone does not establish this premise.*

Proof. Equality elimination transports a proof of $\mathcal{C}(e)$ to a proof of $\mathcal{C}(e')$. Both the equality and the transported proof are checked at their exact types. For the second assertion, two functions on naturals can agree on any fixed finite set while differing at a natural outside it. Thus finite agreement does not imply the equality required by the first assertion. $\square$

For dependent subterms, the surrounding context may itself require transport. The implementation must reconstruct that transport rather than merely substitute syntactically. A domain-specific pruning theorem may be weaker than full equality and still sufficient for a particular contract; its assumptions must be attached to the pruning action.

11.5 What a negative result can mean

Three forms of negative evidence deserve separate treatment. A Lean proof of

$$(\Pi f : \mathsf{T.C}(f) \rightarrow \mathsf{False})$$

shows that no term of the requested type can satisfy the contract under the stated assumptions. A proof about a finite reified grammar shows only that no candidate in that grammar satisfies it. A terminating propositional search can establish nonderivability in its own logic, but that is not automatically a Lean proof of negation.

For example, constructive propositional search may fail to derive excluded middle as a uniform principle. Lean with admitted classical reasoning can nevertheless prove it. Returning “uninhabited in Lean” from the constructive search would be wrong. Similarly, abstracting an inhabited type to an opaque propositional atom cannot support a general negative result merely because the abstract search has no constructor for that atom.

By contrast, the following precise polymorphic negative statement has an elementary checked proof:

```
theorem noPolyCast :
  ((alpha : Type) -> (beta : Type) -> alpha -> beta) -> False := by
  intro f
  exact Empty.elim (f Unit Empty ())
```

It specializes the purported function to `Unit` and `Empty`. This theorem is included in the native experiment. The prototype’s unsuccessful bounded search for such a function is not what proves the theorem; the explicit argument does.

12 An exact contract domain: affine list folds

This section develops a small synthesis domain in full, so that the contract architecture has an executable and mathematically transparent instance. It is intentionally narrower than general program synthesis.

12.1 Candidate schema and specification family

For natural parameters c, a, b, d , define

$$\begin{aligned} F_{c,a,b,d}([]) &= c, \\ F_{c,a,b,d}(x :: xs) &= ax + bF_{c,a,b,d}(xs) + d. \end{aligned}$$

The target family is

$$S_{A,B,C}(xs) = A \sum xs + B |xs| + C, \quad A, B, C \in \mathbb{N}.$$

Length, sum, a weighted combination of the two, and a constant offset are all instances. The synthesis problem is to choose the four implementation parameters so that $F_{c,a,b,d} = S_{A,B,C}$ on all finite lists of naturals.

Theorem 12.1 (Exact affine-fold certificate). *For natural parameters, the following are equivalent:*

1. $F_{c,a,b,d}(xs) = S_{A,B,C}(xs)$ for every finite list xs .
2. The five equations

$$c = C, \quad a = A, \quad bA = A, \quad bB = B, \quad bC + d = B + C \quad (1)$$

hold.

3. The two functions agree on the five lists

$$[], \quad [0], \quad [1], \quad [0, 0], \quad [0, 1].$$

Proof. Clearly (1) implies (3). Assume (3). The empty list gives $c = C$. The list $[0]$ then gives $bC + d = B + C$. For $[1]$, we obtain

$$a + bC + d = A + B + C,$$

so cancellation gives $a = A$. Since $F([0]) = B + C$, evaluation on $[0, 0]$ gives

$$b(B + C) + d = 2B + C.$$

Substituting $bC + d = B + C$ and canceling gives $bB = B$. Finally, $F([1]) = A + B + C$, and evaluation on $[0, 1]$ gives

$$b(A + B + C) + d = A + 2B + C.$$

Using the already established equations reduces this to

$$bA + 2B + C = A + 2B + C,$$

whence $bA = A$. Thus (3) implies (2).

Assume (2). The empty-list equality is $c = C$. For the induction step, suppose $F(xs) = A \sum xs + B |xs| + C$. Then

$$\begin{aligned} F(x :: xs) &= ax + b(A \sum xs + B |xs| + C) + d \\ &= Ax + (bA) \sum xs + (bB) |xs| + (bC + d) \\ &= Ax + A \sum xs + B |xs| + B + C \\ &= A \sum (x :: xs) + B |x :: xs| + C. \end{aligned}$$

This proves (1) by induction. □

The implication $(2) \Rightarrow (1)$ is formalized in the supplied Lean theorem `Leant2Affine.certify`. The other implications are proved above and checked exhaustively by Python on the finite experimental parameter domain; their unrestricted natural-parameter statements are not formalized in the supplied file.

12.2 Why this domain fits Leant2

The certificate is not a statistical assertion. The five equations are finite obligations whose satisfaction yields a proof for lists of arbitrary length and arbitrary natural entries. A parameter solver can search or solve these equations without enumerating large recursive expression trees. The proof reconstruction instantiates one generic induction theorem.

The domain also has a complete counterexample family. Any candidate failing its intended universal contract differs on at least one of the five lists. A counterexample-guided loop can therefore obtain a real separating input rather than treating a failed proof attempt as evidence of incorrectness. This completeness is specific to the affine-fold schema; changing the candidate grammar requires revisiting the theorem.

There can be several correct implementations. For a zero target, many choices of the recurrence multiplier are irrelevant when the base and added contributions remain zero. Therefore, the solver is not expected to recover a unique parameter tuple. It returns the first correct candidate according to the explicitly stated ordering.

12.3 The executable experiment

The Python experiment uses all $3^4 = 81$ candidate tuples with $c, a, b, d \in \{0, 1, 2\}$. It orders them by

$$(c + a + b + d, c, a, b, d).$$

There are $3^3 = 27$ target tuples (A, B, C) in the same coefficient range. For each target, the baseline examines candidates in this order and runs the five-equation certificate check until the first success.

The counterexample-guided version maintains a bank of separating lists. It cheaply skips candidates that disagree with an existing observation. For a remaining candidate, it tries the certificate. If the certificate fails, it scans the fixed five-list family, stores the first mismatch, and continues. If it succeeds, it emits the parameter tuple and a Lean theorem instantiating `certify`.

All arithmetic is exact integer arithmetic. The finite domain and loop order are deterministic. The full traces, certificate equations, observation bank, candidate counts, and generated theorem file are included in `experiments/results.json` and the accompanying source.

The 1,093 held-out lists have lengths zero through six and entries in $\{0, 2, 5\}$. They are a separate implementation check. Universal correctness comes from the certificate theorem, not from these tests. Likewise, the reduction from 1,104 to 105 certificate attempts does not establish a corresponding runtime speedup: the guided method performs additional point evaluations and bookkeeping.

12.4 Lean proof reconstruction

The generic theorem has exactly the five equations as premises. After substituting $a = A$, its proof inducts on the list, expands the recurrence, rewrites the induction hypothesis and multiplier equations, and closes the remaining arithmetic with `omega`. Each generated theorem then uses:

Measured quantity	Result
Candidate grammar size	81
Target specifications solved	27 / 27
Baseline certificate attempts	1,104
Counterexample-guided certificate attempts	105
Replayed counterexamples added across tasks	78
Largest bank reached for a task	4
Candidate-specification pairs exhaustively cross-checked	2,187
Pairs satisfying the universal certificate equations	31
Other pairs with an explicit separating list	2,156
Held-out lists per selected candidate	1,093
Held-out evaluations across selected candidates	29,511

Table 3: Deterministic affine-fold experiment. Certificate-attempt counts are operation counts, not elapsed-time measurements or a comparison with Leant/Djex.

```
-- Example of a generated universal certificate.
theorem length_contract : forall xs : List Nat,
  affine 0 0 1 1 xs = 0 * xs.sum + 1 * xs.length + 0 := by
  apply certify <;> decide
```

The actual generated file contains named theorems `task_000` through `task_026`. In the successful remote run, the generic theorem and all twenty-seven instances reported only `propext` and `Quot.sound` as axioms. No custom axiom, `sorryAx`, or classical choice dependency appeared.

A failed earlier proof script is retained in the development history. It rewrote `a` inside the recursive expression before the induction hypothesis matched, causing a downstream arithmetic failure and recovery placeholders. Substituting `a` before induction repaired the script. The failed run was not counted as accepted evidence.

13 Native Lean experiments and verification results

13.1 What was implemented

The native prototype is a small `MetaM` search procedure, not a Python simulation of Lean typing. It supports function introduction, reflexivity, application of local providers, explicitly selected global providers with fresh universe metavariables, target-constructor application, and continuation-aware rollback. It has no approximate Haskell type representation.

The final standalone file is `prototype/Combined.lean`. Its search code calls native operations such as `whnf`, `MVarId.intro1`, `MVarId.apply`, `MVarId.refl`, `mkConstWithFreshMVarLevels`, and state `save/restore`. The `leant2_with` prototype deliberately accepts exact global names rather than implementing full namespace-aware frontend resolution. That limitation is documented rather than hidden in the examples.

The procedure has a depth/fuel limit, not the production scheduler’s complete work-accounting discipline. It catches failed branch attempts and tries alternatives. It does not implement a persistent provider index, arbitrary case discovery, a verified negative-search calculus, fair interleaving, learned conflicts, automatic invariants, or a hardened standalone acceptance service. Ordinary Lean elaboration and compilation check the generated declarations in this experiment.

13.2 The final native corpus

Fourteen definitions are fully synthesized by the prototype: identity, composition, the S combinator, pair construction with swapped fields, dependent function application, dependent-pair packing, a simple rank-two target, a successor satisfying a universal contract, a natural witness constrained to be one, an indexed vector constructor, use of a local `Inhabited` instance, contract-directed selection between same-typed payloads, a behavior-constrained rank-two function, and a function that constructs a polymorphic callback.

Two further definitions use manually supplied templates: vector mapping supplies the structural match and smaller recursive call, and equality transport supplies a case split on the equality. Their remaining branch terms are synthesized. The helper that converts a vector to a list is written directly. Four audited theorems state identity behavior, successor behavior, vector-map behavior, and impossibility of the unrestricted polymorphic cast.

Thus the final combined run audits twenty named declarations: sixteen synthesized or template-assisted definitions and four theorems. Nineteen report no axioms. The vector-map behavior theorem reports only `propext`. The run evaluates the successor at 12 to 13, the constrained natural witness to 1, the selected right payload to 9, and a two-element mapped vector to the list `[3, 4]`.

A rank-two type alone does not enforce callback use. The simple `rankTwo` case tests acceptance of the higher-rank target shape, but it could be solved by a behaviorally uninteresting function. The separate `rankTwoSpec` case forces agreement with application of the supplied polymorphic function. This distinction prevents a trivial inhabitant from being mislabeled as evidence for intended higher-order behavior.

13.3 Remote compilation protocol

The local container had no Lean compiler and could not resolve external hosts. The attached tools documented AXLE’s checking endpoint, so generated Lean source was sent through a synchronous Wolfram HTTP request to AXLE. Requests specified environment `lean-4.34.0`, retained the actual imports, and checked definitions as well as theorems. The returned source was compared with the submitted source modulo final whitespace, and `Lean.versionString` returned `4.34.0`. The service’s documentation distinguishes compilation status from proof acceptance [12].

The final native run’s request identifier is `ca4a866a-9382-4e0d-aa57-bae130d2f51c`. It returned `okay=true`, no failed declarations, and no Lean errors. It retained two nonfatal linter warnings: the negative `fail_if_success` harness deliberately leaves no proof-state change, and the left payload is an intentionally available alternative that the final chosen term does not use explicitly. The service also issued advisory warnings about using `import Lean` instead of its preferred Mathlib header. Imports were not overridden.

The affine certificate run’s request identifier is `4e277309-c66b-4cc3-bd76-74a1f2153b02`. It returned `okay=true`, no failed declarations, no Lean errors, and no Lean or service warnings. Both final receipts include the reported executor identity. Their server timings are preserved for provenance, but are not presented as comparative synthesis benchmarks.

13.4 A useful failed attempt: recursor versus executable form

The first combined vector attempt used tactic induction directly to construct the computational definition. The axiom audit of that definition was empty, but the code generator reported that it did

not support the resulting `Vec.rec` form. The associated behavior proof and evaluation also failed in that run.

The final file changes the implementation to a structural pattern match. In the cons branch it binds the explicitly smaller call and lets the prototype fill the result. This version compiles, its behavior proof checks, and its evaluation succeeds. The development history records both outcomes. The lesson is architectural: logical inhabitation and code-generation acceptance are different gates, even when the intended function is elementary and total.

13.5 Negative integrity controls

Control	okay	Observed reason for rejection
Use <code>True.intro</code> as a proof of <code>False</code>	false	Type mismatch; declaration failed.
Prove identity sends zero to one by reflexivity	false	Reflexivity failed; declaration failed.
Declare a custom axiom of <code>False</code> and use it	true	Failed-declaration report and axiom audit expose <code>forged</code> .
Use <code>sorry</code> to prove <code>False</code>	true	Failed-declaration report, warning, and axiom audit expose <code>sorryAx</code> .

Table 4: All four controls are rejected by the stated acceptance policy. The last two demonstrate why compilation status alone is insufficient.

The negative-source files are deliberately isolated under `experiments/negative_controls`. They are not imported by the positive prototype or certificate files. A successful run of these controls means that their expected rejection behavior was observed, not that any false proposition was accepted as a valid theorem.

The native corpus also contains a bounded synthesis-failure harness for an unrestricted polymorphic cast. Its failure is an empirical search outcome. The separately proved `noPolyCast` theorem supplies the actual mathematical negative evidence.

13.6 What the evidence does not establish

The experiments do not measure success on the full Leant or Djex corpora. They do not show that the new system is faster, that it covers a given percentage of practical Lean types, or that the proposed scheduler scales to a large Mathlib environment. The Python search experiment is an executable model of a semantic domain, not a production Lean implementation of that domain. The remote service did not provide an exact Mathlib revision, and no independent checker was run.

What is established is narrower and concrete: exact Lean metaprogramming can implement the chosen shared-continuation search mechanism; several universe-polymorphic and dependent examples compile; contract constraints can force backtracking over program values; an executable structural template can be completed; a small semantic synthesis domain yields checked universal proofs; and negative controls distinguish superficial compilation from accepted evidence.

14 Concrete implementation organization

14.1 Module boundaries

The main library should depend initially on Lean and its standard library. Proof backends that require additional project libraries are optional imports. This keeps basic synthesis useful outside Mathlib while allowing a richer mathematical profile inside it. The proposed module layout is:

```
Leant2/
  Frontend/      -- commands, terms, tactics, editor suggestions
  Query/         -- frozen elaboration, contexts, policy, identities
  Native/        -- version-specific Lean API adapters
  Index/         -- providers, constructors, recursors, contracts
  Search/        -- holes, dependencies, rules, continuations, queues
  Rules/         -- introductions, spines, cases, transport
  Recursion/     -- descriptors, carriers, structural/WF templates
  Contracts/     -- exact predicates, examples, backward rules
  Domains/       -- length, affine folds, finite data, bitvectors
  Proof/         -- core automation and optional tactic adapters
  Verify/        -- closed declarations, statement and axiom audit
  Export/        -- checked plans, presentation, replay receipts
  Worker/        -- project-local protocol, resources, isolation
  Tests/         -- semantic regressions and negative controls
```

The version-sensitive layer is intentionally small. It wraps native context operations, goal construction, constructor metadata, instance synthesis, dependent elimination, and final declaration checking. Search policies should not scatter assumptions about Lean internal APIs throughout the codebase.

The existing proof automation and semantic libraries should be reused through adapters, not copied into a second quasi-standard library. Relevant concrete facilities are Lean’s expression and metavariable operations, persistent environment extensions and discrimination-style indexes, constructor and recursor metadata, ordinary simplification and decision procedures, and project-enabled Aesop or arithmetic tactics. Exact API names outside the compiled prototype should be confirmed against the selected toolchain when implemented.

14.2 Rule and worker interfaces

A production rule should expose bounded resumable work, rather than a function that can monopolize the request until it succeeds. One schematic interface is:

```
Rule.prepare : FrozenQuery -> HoleView -> MetaM RuleCursor
Rule.step    : RuleCursor -> SliceBudget -> MetaM RuleStep

RuleStep :=
  | alternative CheckedConstructionPlan NewObligations Continuation
  | suspended RuleCursor ResourceReport
  | exhausted ScopeReport
  | unsupported Reason

Verifier.check : FrozenQuery -> FrozenCandidate -> VerifyBudget
               -> VerificationOutcome
```

“Checked construction plan” here means checked as an intermediate rule application in its owning context. It is not yet an accepted solution. Only the verifier can create the final certified-result authority. Workers exchange request identifiers, environment identities, serialized query handles, plans or closed terms, and structured diagnostics. Every response identifies its owning request and candidate. Stale responses from a canceled request are discarded. A protocol parser is bounded in input size, nesting, and message count. Neither a log line nor a solver-supplied success tag is a proof artifact.

14.3 A complete finalization algorithm

The implementation should make finalization easy to inspect:

```
finalize(Q, proposedProgram, proposedProof):
  require Q is still the active frozen query
  freeze and close program under Q.telescope
  reject unresolved term/universe metavariables or escaping locals
  check program at the exact frozen target in a scratch environment
  freeze and close proof against Q.contract applied to that program
  check proof in the same controlled environment
  audit both dependency closures and all introduced declarations
  enforce Q.assumptionPolicy
  if Q.requiresExecutable:
    compile the exact program and check Q.effectPolicy
  export only after the transaction succeeds
  optionally re-elaborate displayed source and compare its meaning
```

This order prevents a contract that ignores its argument from hiding an invalid program. It also prevents the proof backend from changing the frozen statement by resolving leftover input metavariables. A separate high-assurance deployment can replay exported declarations against a challenge in an independent checker process.

For typed-only mode, omit the behavioral proof step but retain the program, environment, and policy checks. For test-consistent mode, record every checked or evaluated observation and its assurance level. The same object must not be reported as certified merely because its status passed through the typed-only path first.

14.4 Initial operational profiles

Use named profiles rather than many unexplained numeric knobs. A *local* profile searches locals, constructors, projections, and explicitly supplied providers with small type-instantiation and case budgets. A *library* profile adds indexed provider composition and selected proof automation. A *recursive* profile enables registered templates and verified summaries. An *audit* profile uses a fixed explicit grammar, deterministic enumeration, no silent beam eviction, and complete discarded-work accounting.

Each profile still has total heartbeats or equivalent work, wall time, memory, generated-node count, candidate count, proof-worker limits, and cancellation behavior. Default numerical values should be tuned from actual benchmark measurements rather than invented from the tiny experiment here. The prototype’s fuel values of 6, 24, and 32 are experimental depth limits only; they are not proposed production defaults.

14.5 Errors should explain the missing mechanism

A useful failure report identifies the best partial constructions, the blocking obligations, the provider scope searched, and which limits were reached. For instance:

```
No certified result within this request.
Native constructor/application search: exhausted at depth 12.
Recursive templates tried: list fold, generalized accumulator.
Best typed candidate: retained for inspection.
Remaining obligation: accumulator invariant preservation.
Verification: arithmetic backend timed out; no counterexample.
Negative conclusion about all Lean inhabitants: not established.
```

This is proposed diagnostic text. It is more actionable than a generic “unsupported dependent type” message and more honest than “no implementation exists.” The report should not invent a human explanation from opaque failure codes; rule adapters return structured reasons that the renderer can summarize.

15 A development sequence with measurable gates

15.1 Stage I: freeze, construct, and reject correctly

Build the query owner, exact local telescope handling, native introduction/application/constructor rules, and finalizer. The acceptance gate is not just a set of successful examples. It includes wrong-target, wrong-instance, unresolved-metavariable, escaping-local, hidden-sorry, custom-axiom, and stale-environment controls. The first public form can be a tactic that replaces a term hole and a command that emits a definition plus its contract theorem.

The supplied native core is a useful starting experiment, not a production module to adopt unchanged. Before adoption it needs complete resource accounting, structured failures, qualified-name resolution, explicit root validation, and ownership tests. Its shared-continuation mechanism should remain visible and testable as those features are added.

15.2 Stage II: native dependency as a first-class feature

Add result-directed provider indexes, full dependent spines, contextual universe tests, local-instance fidelity, dependent-pair search, indexed constructors, and equality transport. The gate contains at least one positive and one negative case for each kind of dependency. Tests should include two locals with identical types but different values, two class dictionaries with distinct computational payloads, and multiple universes whose roles cannot be swapped.

A successful signature alone is insufficient when there are trivial inhabitants. Every behavioral family should have contracts that distinguish its intended provider usage or data flow. The separate higher-rank contract in the prototype illustrates this requirement.

15.3 Stage III: dependent elimination and structural recursion

Implement the motive/generalization adapter and a small recursion-descriptor library for lists, trees, and vectors. Begin with fixed public templates, then add demand-derived carriers and generalized accumulator parameters. The gate requires executable code, termination checking, exact result indices,

and checked behavioral theorems. A manually provided template and an automatically discovered template are separate benchmark categories.

Regression cases should include vector map, vector append with any necessary index equality transport, a tree fold, a list left-fold expressed through a function carrier, and accumulator reverse. Each tests a different failure mode. Passing vector map does not establish support for arbitrary indexed recursion.

15.4 Stage IV: contract domains and proof integration

Port the affine-fold parameter search from the Python experiment into a Lean domain plugin, keeping its generic certificate theorem. Add a length-summary domain and a finite-data domain with independently checked encodings. Introduce the three-valued verifier interface and fair revisiting of unknown candidates.

The gate requires exact Lean replay of every rejected model used for proof-authorized pruning, proof reconstruction for every certified answer, and spurious-model controls for deliberately coarse abstractions. Measure both certificate calls and end-to-end time; reductions in one cannot stand in for the other.

15.5 Stage V: scale and assurance

Add persistent indexes, scoped positive caches, bounded conflict learning, abstraction-guided provider composition, and parallel workers. Only after the serial invariants are stable should the system share validated cache entries across workers. CI should exercise request cancellation, worker replacement, malformed outputs, stale results, and environment changes.

At this stage, add independent declaration replay and challenge comparison for higher-assurance deployments. A theorem-producing library plugin should not be able to redefine the user's proposition or add a hidden axiom merely by changing the worker environment. Logical correctness and operating-system isolation need separate tests.

15.6 Stage VI: broaden deliberately

Well-founded recursion, automatically synthesized invariant families, quotient-respecting constructions, and effectful programs should be added as explicit capabilities with individual acceptance gates. For effectful programs, a verified weakest-precondition interface is preferable to executing arbitrary IO candidates during search. Lean's current `mvccgen` facility is a relevant integration point for supported monadic verification, not a claim that arbitrary effects become synthesizable [13].

The expansion criterion is evidence of a reusable mechanism on held-out tasks. Merely adding many one-off templates can improve a benchmark count without improving the architecture. The public documentation should distinguish built-in templates, library reuse, and genuine open-ended term construction.

16 Evaluation: making practical coverage meaningful

16.1 Separate tasks that look superficially similar

A useful evaluation suite has several independent tracks. Type inhabitation asks for any admissible term. Behavioral synthesis asks for a specified implementation. Library composition allows the intended operation as an available component. Rediscovery withholds that operation and relevant aliases.

Template completion supplies a recursion structure. Template discovery does not. Proof repair and program repair are separate tasks with different allowed edits.

Reporting a single success percentage across these tracks would obscure what the system actually does. In particular, `fun _ => []` may solve an unconstrained list-to-list type but should not count as successful list processing in the behavioral track.

16.2 Coverage matrix

Family	Proposed benchmark focus	Evidence in this package
Polymorphic plumbing	Several universes, nested callbacks, explicit and implicit binders	Small compiled positive corpus.
Dependent pairs and records	Earlier-field choices affect later fields and contracts	Sigma and payload examples.
Indexed constructors	Inferred parameters and nontrivial index constraints	Vector cons example.
Dependent elimination	Motives, dependent locals, impossible branches	Hand-guided equality transport; broader adapter proposed.
Structural recursion	Smaller calls, generalized parameters, executable export	Hand-supplied vector-map template with synthesized branches.
Contract domains	Exact bridge, certificates, replayed counterexamples	Affine-fold search plus 27 universal proofs.
Large library composition	Retrieval quality, opaque/reducible heads, provider diversity	Proposed, not benchmarked here.
Well-founded recursion	Measures and decrease proofs	Proposed.
Effects and quotients	WP or quotient-respect obligations	Proposed later capabilities.

16.3 Benchmark provenance and held-out evaluation

Reuse suitable existing Leant/Djex examples as a compatibility corpus, but pin their source revisions and do not mix historical receipts with new runs. Add native Lean cases that the old fragment representation necessarily treats conservatively. Select held-out tasks from actual user modules after removing names or declarations that directly solve the query, according to the track’s rules.

Record exact imports, toolchain, provider inventory, type and behavior constraints, trust policy, resource limits, and allowed templates. Development examples should not be presented as held-out tests. Family-level splits are preferable to merely renaming identifiers in near-duplicate tasks. Performance results should be repeated enough to separate warm-cache behavior, compilation overhead, and actual search variation.

16.4 Metrics beyond solved count

Measure time to first typed candidate, first test-consistent candidate, and first certified candidate separately. Record peak memory, expanded native states, unification and normalization effort, provider candidates retrieved, proof attempts, counterexample replays, compilation failures, and final proof

size. Candidate quality includes readability and dependency footprint, but these must not override correctness.

Ablations should remove shared-continuation backtracking, dependent result matching, carrier generalization, semantic filtering, and cache layers one at a time. The payload and constrained-natural examples provide small regression witnesses for backtracking. Larger workloads are needed to judge its performance cost. The affine experiment provides a domain-level comparison of certificate attempts, not an end-to-end Leant2 ablation.

For negative results, count separately: checked impossibility theorems; exhaustive rejection of a finite candidate grammar; exhaustive failure of a bounded proof grammar; and ordinary timeout. Even a finite program grammar does not by itself make arbitrary universal contracts decidable. Enumerating all its candidates while leaving some proofs unknown cannot establish that the grammar contains no correct program.

17 Design alternatives and final recommendation

A direct Haskell-to-Lean translation would preserve too much of the original approximation boundary. A tactic-only wrapper would delegate useful proof obligations but provide too little control over program alternatives, recursion structures, and behavioral ranking. A solver-only design would either restrict the specification language severely or place an unsafe semantic translation at its center. A giant unrestricted enumerator would retain exactness but squander the information already present in dependent types and contracts.

The recommended design combines a small exact native core with explicitly scoped accelerators. Its decisive engineering commitments are a frozen query, one owner for contextual semantics, continuation-aware program/proof search, exact dependent spines and motives, restricted recursive-call capabilities, proof-producing contract domains, and an acceptance transaction independent of heuristic success. Resource limits and incomplete search are normal operational outcomes, not exceptions to hide.

The experiments justify implementing the first stages: the native core already handles representative polymorphic and dependent constructions, shared backtracking resolves behavioral choices, structural-template completion can produce executable code, and a small domain can reconstruct universal proofs from synthesized parameters. They also identify two immediate safeguards: compiler acceptance must be checked separately from logical typing, and a reported compilation success must be combined with declaration and axiom audits.

The resulting Leant2 would have a credible route toward broad practical usefulness without claiming to solve an undecidable general problem. Its strength would come from using Lean’s full semantic structure where it is available, and from making every shortcut, unsupported case, and completeness claim explicit.

A Repository source map and version scope

The Leant commit inspected was `6bf05ad78c467989e68290f2d08bbbed40802d485`. Its pinned Djex submodule was `e237e8667190aff38faaa590ce5b12afbfffac452`. The review used repository README material and these focused source/document ranges:

Path and reviewed range	Design relevance
Leant <code>src/Leant/Synth/Fragment.hs</code> , lines 1–160	Exact retained fragment versus opaque dependent subterms; restrictions on negative conclusions.
Leant <code>src/Leant/Synth/Behavioral.hs</code> , lines 1–180	Same-candidate behavior checks, candidate retention, proof/refutation/unknown distinction.
Leant <code>docs/synth-internals.md</code> , lines 1–220	Ownership of semantic origin and candidate evidence.
Djex <code>docs/architecture.md</code> , lines 1–165	Shared foundation, checked adapters, ownership and component layout.
Leant <code>docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex</code> , lines 1–260	Earlier feasibility and worker-boundary proposal, used as a starting point rather than a new contribution.

These ranges support the architectural observations stated here; this is not a claim of having audited every line of either repository. The existing rewrite report itself uses older historical snapshots, which are not substituted for the current review baseline.

The Lean documentation references use their official rolling endpoints as accessed on September 17, 2026. The experiments are specifically pinned to the remotely reported Lean 4.34.0 environment. The code generator behavior and tactic outcomes reported in Section 13 concern that environment, not every Lean release.

B Artifact guide and reproduction

The archive contains the complete LaTeX source tree and compiled PDF under `article/`. The main source is `article/leant2.tex`. It requires a conventional LaTeX installation with the packages named in its preamble and can be built with `latexmk -pdf leant2.tex` from that directory.

The primary native test is the standalone `prototype/Combined.lean`, exactly as submitted in the final successful run. `NativeCore.lean` and `DependentExamples.lean` provide a modular view; the latter imports the former. The combined file is the authoritative remotely checked artifact. The local modular build arrangement was not separately run in this session.

Run `python3 experiments/run_experiments.py` from the archive root to regenerate the deterministic experiment data and `Certificates.lean`. The generic certificate prelude is an intentionally open namespace fragment; it is consumed by the generator rather than compiled alone. The generated `experiments/Certificates.lean` is self-contained and requires a compatible Mathlib environment. The service did not expose an exact Mathlib commit, so this archive does not invent one.

The optional `experiments/check_axle.py` client sends a specified source file to the same endpoint using Python’s standard library and stores the raw response in a user-selected output file. It needs network access and does not by itself constitute a high-assurance verifier. Its conservative status report checks compilation, failed declarations, and required axiom-audit messages. Future local or remote reruns can differ when the service or dependencies change.

Normalized receipts under `experiments/receipts/` retain request identifiers, source identities, observed audit outcomes, warnings, and development failures. They are transcriptions of the actual

reported results, not original raw response bytes. Deliberately invalid controls are isolated under `experiments/negative_controls/`; they must not be included in an ordinary positive build.

C The minimal continuation mechanism

The following pseudocode extracts the key mechanism from the compiled prototype. The actual Lean source contains the API details and test corpus.

```
search(fuel + 1, goal :: rest):
  checkpoint := saveState()
  attempt(action):
    restore(checkpoint)
    try:
      children := action(goal)
      if search(fuel, children ++ rest):
        return true
    catch branchFailure:
      pass
    restore(checkpoint)
    return false

try function introduction when the target is a Pi
try reflexivity
try each local provider
try each explicitly admitted global provider
try each constructor of the target inductive
restore(checkpoint)
return false
```

The consequential line is `children ++ rest`. It keeps later dependent obligations inside the success condition of an earlier choice. Replacing it with “solve children, return their first answer, then solve rest elsewhere” loses this behavior unless the surrounding system explicitly preserves and resumes those alternatives.

The minimal code is deliberately not a recommended final resource scheduler: recursive fuel bounds derivation depth but does not bound all primitive work, and broad exception catching is too coarse for production diagnostics. The full design adds per-action limits, resumable cursors, dependency support, separate acceptance, and honest unknown results around this same mechanism.

References

- [1] Vladimir Reshetnikov and contributors. *Djex architecture*. Commit e237e8667190aff38faaa590ce5b12afbfffac452. <https://github.com/VladimirReshetnikov/Djex/blob/e237e8667190aff38faaa590ce5b12afbfffac452/docs/architecture.md>.
- [2] Vladimir Reshetnikov and contributors. *Leant synthesis fragment translator*, `src/Leant/Synth/Fragment.hs`. Commit 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/src/Leant/Synth/Fragment.hs>.
- [3] Vladimir Reshetnikov and contributors. *Leant behavioral proof checks*, `src/Leant/Synth/Behavioral.hs`. Same Leant commit. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/src/Leant/Synth/Behavioral.hs>.
- [4] *Rewriting Leant and Djex in Lean 4: Feasibility, Architectural Consequences, Tradeoffs, and Recommended End State*. Existing Leant project report, dated August 15, 2026; source read from the review commit above. https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/docs/Leant_Djex_Lean4_Rewrite_Analysis/Leant_Djex_Lean4_Rewrite_Analysis.tex.
- [5] Lean development team. *The Lean Language Reference: Validating a Lean Proof*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [6] Lean development team. *The Lean Language Reference: Universes*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/Universes/>.
- [7] Lean development team. *The Lean Language Reference: The Type System*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/>.
- [8] Aesop contributors. *Aesop: automated proof search for Lean*, project README and rule documentation. Accessed September 17, 2026. <https://github.com/leanprover-community/aesop>.
- [9] Lean development team. *The Lean Language Reference: The grind tactic*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [10] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. arXiv:1510.08419, version 3, 2016. <https://arxiv.org/abs/1510.08419>.
- [11] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. *Program Synthesis by Type-Guided Abstraction Refinement*. arXiv:1911.04091, 2019; DOI:10.1145/3371080. <https://arxiv.org/abs/1911.04091>.
- [12] Axiom Math. *AXLE documentation: check tool*. Accessed September 17, 2026. <https://axle.axiommath.ai/v1/docs/tools/check/>.
- [13] Lean development team. *The Lean Language Reference: The mvcgen tactic*. Accessed September 17, 2026. <https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/>.